Spring Session

一、 HttpSession 回顾

1 什么是 HttpSession

是 JavaWeb 服务端提供的用来建立与客户端会话状态的对象。

二、 Session 共享

1 什么是 Session 共享

是指在一个浏览器对应多个 Web 服务时，服务端的 Session 数据需要共享。

2 Session 共享应用场景

1) 单点登录
2) Web 服务器集群等场景都需要用到

3 Session 共享常见的解决方案

3.1 Session 复制

通过对应用服务器的配置开启服务器的 Session 复制功能，在集群中的几台服务器之间同步 Session 对象，使得每台服务器上都保存所有的 Session 信息，这样任何一台宕机都不会导致 Session 的数据丢失，服务器使用 Session 时，直接从本地获取。这种方式的缺点也比较明显。因为 Session 需要时时同步，并且同步过程是有应用服务器来完成，由此对服务器的性能损耗也比较大。

3.2 Session 绑定

利用 hash 算法，比如 nginx 的 ip_hash,使得同一个 Ip 的请求分发到同一台服务器上。

这种方式不符合对系统的高可用要求，因为一旦某台服务器宕机，那么该机器上的 Session 也就不复存在了，用户请求切换到其他机器后么有 Session，无法完成业务处理。

3.3 利用 Cookie 记录 Session

Session 记录在客户端，每次请求服务器的时候，将 Session 放在请求中发送给服务器，服务器处理完请求后再将修改后的 Session 响应给客户端。这里的客户端就是 cookie。

利用 cookie 记录 Session 的也有缺点，比如受 cookie 大小的限制，能记录的信息有限，安全性低，每次请求响应都需要传递 cookie，影响性能，如果用户关闭 cookie，访问就不正常。

尚学堂·百战程序员

3.4 Session 服务器

Session 服务器可以解决上面的所有的问题，利用独立部署的 Session 服务器统一管理 Session，服务器每次读写 Session 时，都访问 Session 服务器。

对于 Session 服务器，我们可以使用 Redis 或者 MongoDB 等内存数据库来保存 Session 中的数据，以此替换掉服务中的 HttpSession。达到 Session 共享的效果。

三、 什么是 Spring Session

Spring Session 是 Spring 的项目之一。Spring Session 提供了一套创建和管理 Servlet HttpSession 的方案，默认采用外置的 Redis 来存储 Session 数据，以此来解决 Session 共享的问题。

四、 Spring Session 的使用

1 安装 Redis

第一步 需要在 linux 系统中安装 gcc
命令：yum install -y gcc-c++

第二步 需要将下载好的 redis 压缩包添加到 linux 服务器中
版本：redis-3.0.0.tar.gz
redis 的版本：副版本号奇数版本号是测试版，不建议在生产环境中使用。
偶数版本时稳定版建议在生产环境中使用。
3.0 版本更新比较大。集成了集群技术

第三步 解压压缩包
命令：tar -zxvf redis.....

第四步 编译 redis
命令：进入 redis 的解压完毕的根目录下 执行命令：make

第五步 安装 redis
命令：进入 redis 的解压完毕的根目录下，执行命令： make install PREFIX=/usr/local/redis

第六步：启动 redis
1，前端启动
在 bin 目录下执行命令： ./redis-server （ctrl+c）退出 redis

2.后端启动

尚学堂·百战程序员

(1)先将 redis 解压目录下的 redis.conf 文件拷贝到 安装好的 redis 的 bin 目录下
命令：cp redis.conf /usr/local/redis/bin

(2)修改拷贝过来的 redis.conf 配置文件
命令：vim redis.conf
将 daemonize no 改为 yes

(3)启动 redis
在 bin 目录下执行命令：./redis-server redis.conf

(4)查看 redis 启动是否成功
输入命令：ps aux|grep redis

(5) 关闭 redis 的命令
./redis-cli shutdown

第七步：测试 redis
在 bin 目录下启动 redis 自带的客户端 ./redis-cli
常见 redis 命令：
ping--->pong

2 搭建案例环境

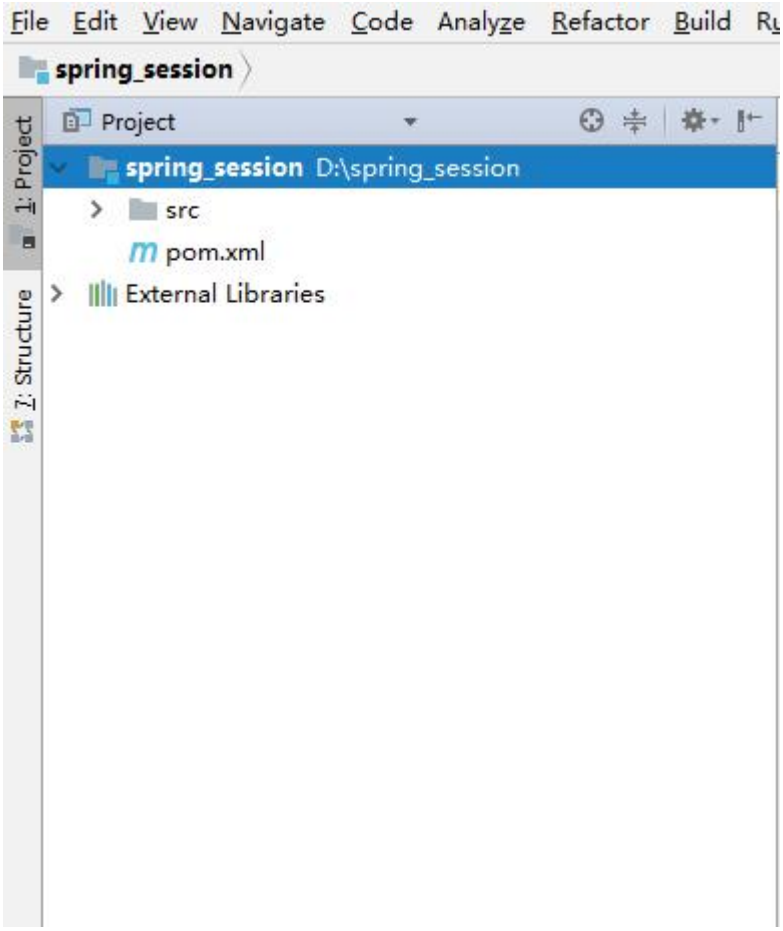
2.1版本介绍

JDK：1.8
Spring Boot：2.1.6.RELEASE
Spring Session：Bean-SR3

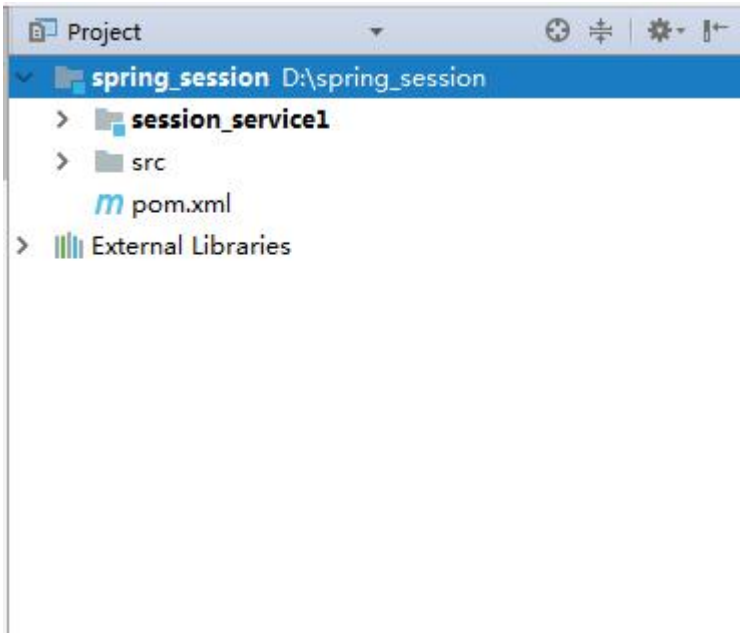
2.2创建项目

创建父工程 spring_session

尚学堂·百战程序员

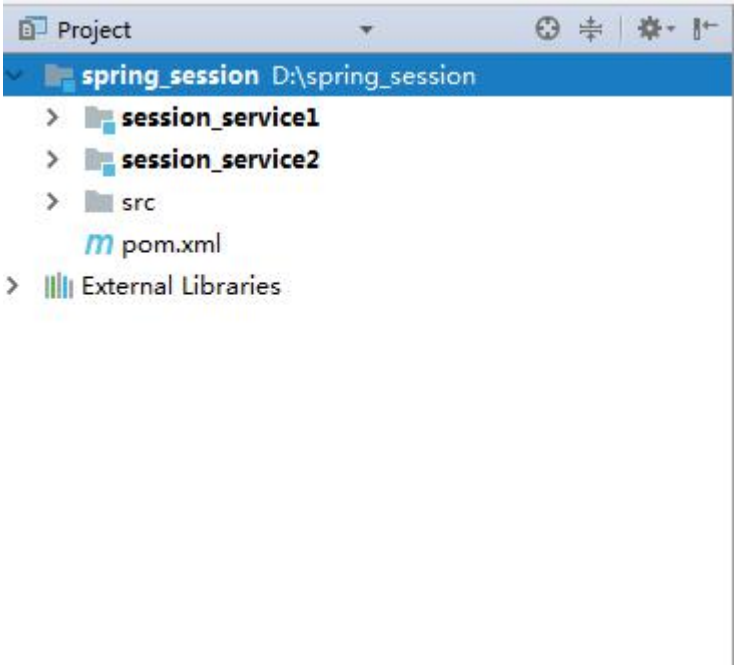


创建模块 session_service1



创建模块 session_service2

尚学堂·百战程序员



2.3 修改 POM 文件添加坐标依赖

修改父工程 POM 文件

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.bjsxt</groupId>
    <artifactId>spring_session</artifactId>
    <packaging>pom</packaging>
    <version>1.0-SNAPSHOT</version>
    <modules>
        <module>session_service1</module>
        <module>session_service2</module>
    </modules>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactid>
        <version>2.1.6.RELEASE</version>
    </parent>
```

尚学堂·百战程序员

```
    <dependencyManagement>
        <dependencies>
            <dependency>

<groupId>org.springframework.session</groupId>
                <artifactId>spring-session-bom</artifactId>
                <version>Bean-SR3</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <dependencies>
        <!--web Starter-->
        <dependency>
            <groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!--Redis Starter-->
        <dependency>
            <groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-redis</artifactId>
        </dependency>

        <!--Spring session data redis-->
        <dependency>
            <groupId>org.springframework.session</groupId>

<artifactId>spring-session-data-redis</artifactId>
        </dependency>


        <!--Lettuce 是一个基于Netty的NIO方式处理Redis的技术-->
        <dependency>
            <groupId>io.lettuce</groupId>
            <artifactId>lettuce-core</artifactId>
        </dependency>
    </dependencies>

</project>
```

尚学堂·百战程序员

3 添加配置文件

3.1 session_service1

```
#服务端口
server:
  port: 8080
spring:
  redis:
    host: 192.168.70.156  #Redis 配置
    port: 6379
    password:
    database: 0
```

3.2 session_service2

```
#服务端口
server:
  port: 8081
spring:
  redis:
    host: 192.168.70.156  #Redis 配置
    port: 6379
    password:
    database: 0
```

4 创建启动类

4.1 session_service1

```
@SpringBootApplication
```

尚学堂·百战程序员

```
@EnableRedisHttpSession
public class Service1Application {
    public static void main(String[] args){

SpringApplication.run(Service1Application.class,args);
    }
}
```

4.2 session_service2

```
@SpringBootApplication
@EnableRedisHttpSession
public class Service2Application {
    public static void main(String[] args){

SpringApplication.run(Service2Application.class,args);
    }
}
```

5 编写测试代码测试效果

5.1 session_service1

5.1.1 创建 Controller

```
@RestController
@RequestMapping("service1")
public class WebController {

    @RequestMapping("/setMsg")
    public String setMsg(HttpSession session,String msg){
        session.setAttribute("msg",msg);
        return "ok";
    }
}
```

尚学堂·百战程序员

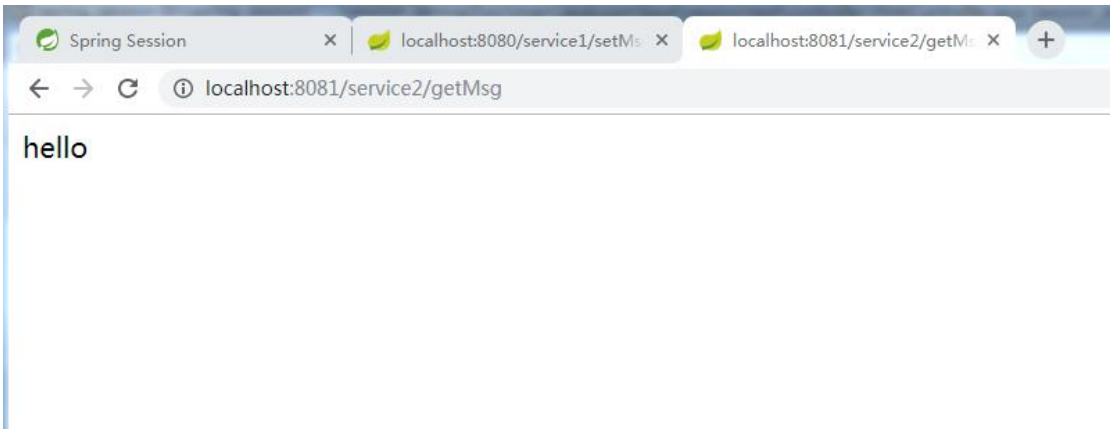
5.2 session_service2

5.2.1 创建 Controller

```
@RestController
@RequestMapping("/service2")
public class WebController {

    @RequestMapping("/getMsg")
    public String getMsg(HttpSession session){
        String msg = (String) session.getAttribute("msg");
        return msg;
    }
}
```

5.2.2 测试



6 共享自定义对象

6.1 创建 Users 实体类

```
public class Users implements Serializable {
    private String username;
    private String userpwd;

    public String getUsername() {
        return username;
    }
```

尚学堂·百战程序员

```
    public String getUserpwd() {
        return userpwd;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public void setUserpwd(String userpwd) {
        this.userpwd = userpwd;
    }
}
```

6.2 修改 session_service1 的 controller

```
@RestController
@RequestMapping("service1")
public class WebController {

    @RequestMapping("/setMsg")
    public String setMsg(HttpSession session,String msg){
        session.setAttribute("msg",msg);
        return "ok";
    }

    /**
     * 获取 Users 信息，保存到 HttpSession 中
     */
    @RequestMapping("/addUsers")
    public String addUsers(HttpSession session, Users users){
        session.setAttribute("u",users);
        return "ok";
    }
}
```

6.3 修改 session_service2 的 controller

```
@RestController
```

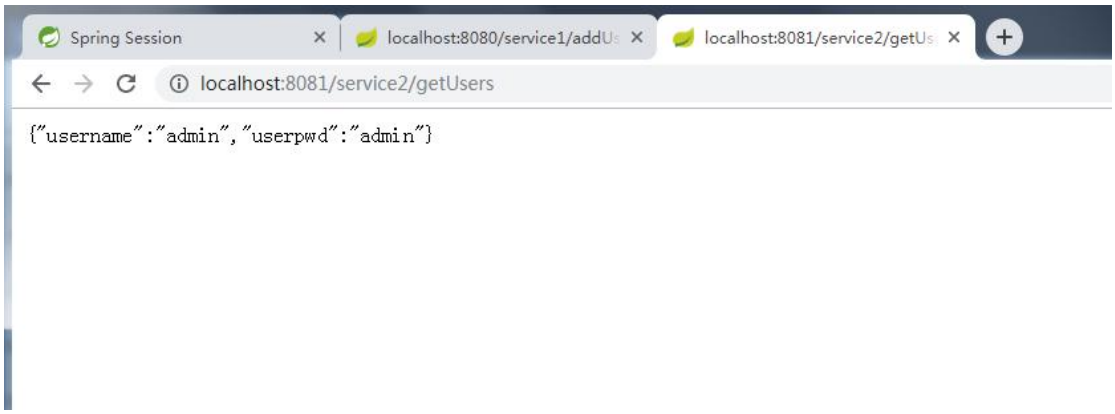
尚学堂·百战程序员

```
@RequestMapping("/service2")
public class WebController {

    @RequestMapping("/getMsg")
    public String getMsg(HttpSession session){
        String msg = (String) session.getAttribute("msg");
        return msg;
    }

    /**
     * 获取 HttpSession 中的 Users 对象
     */
    @RequestMapping("/getUsers")
    public Users getUsers(HttpSession session){
        Users users = (Users) session.getAttribute("u");
        return users;
    }
}
```

6.4测试



7 Spring Session 的 Redis 存储结构

spring:session:expirations:(Set 结构)用户 ttl 过期时间记录
这个 k 中的值是一个时间戳，根据这个 Session 过期时刻滚动至下一分钟而计算得出。
这个 k 的过期时间为 Session 的最大过期时间 +5 分钟。

spring:session:sessions:(Hash 结构)

maxInactiveInterval：过期时间间隔

creationTime：创建时间

尚学堂·百战程序员

lastAccessedTime：最后访问时间

sessionAttr：Attributes 中的数据

存储 Session 的详细信息，包括 Session 的过期时间间隔、最后的访问时间、attributes 的值。这个 k 的过期时间为 Session 的最大过期时间 +5 分钟。

spring:session:sessions:expires:(String 结构)过期时间记录

这个 k-v 不存储任何有用数据，只是表示 Session 过期而设置。

这个 k 在 Redis 中的过期时间即为 Session 的过期时间间隔。

8 设置 Session 的失效时间

8.1 在注解中设置失效时间

```
@SpringBootApplication
@EnableRedisHttpSession(maxInactiveIntervalInSeconds=10)
public class Service1Application {
    public static void main(String[] args){

SpringApplication.run(Service1Application.class,args);
    }
}
```

9 @EnableRedisHttpSession 注解讲解

9.1 maxInactiveIntervalInSeconds

设置 Session 的失效时间，单位为秒。默认(1800 秒)30 分钟。

9.2 redisNamespace

为键定义唯一的命名空间。该值用于通过更改前缀与默认 spring:session 隔离会话

9.3 redisFlushMode

Redis 会话的刷新模式。默认值为“保存”

9.4 cleanupCron

过期会话清理作业的 cron 表达式。默认值("0 * * * * *")每分钟运行一次。

尚学堂·百战程序员

10 更换 Spring Session 的序列化器

Spring Session 中默认的序列化器为 jdk 序列化器，该序列化器效率低下，内存再用大。我们可以根据自己的需要更换其他序列化器，如 GenericJackson2JsonRedisSerializer 序列化器。

10.1在配置类中创建自定义序列化器

```
@Configuration
public class SpringSessionConfig {

    /**
     * 更换默认的序列化器
     *
     */
    @Bean("springSessionDefaultRedisSerializer")
    public RedisSerializer defaultRedisSerializer(){
        return getSerizlizer();
    }
    /**
     * 定义序列化器
     */
    private RedisSerializer getSerizlizer(){
        return new GenericJackson2JsonRedisSerializer();
    }
}
```

10.2测试

尚学堂·百战程序员